PYTHON PROGRAMMING

MODULE1

Introduction, Variables, ExpressionsandStatements, Iterations and Functions

MODULE1

Python:Introduction,Variables,ExpressionsandStatements, Iterations and Functions

Syllabus:

- **Thewayoftheprogram:**ThePythonprogramminglanguage,whatisa program? What is debugging? Syntax errors, Runtime errors, Semantic errors, Experimental debugging.
- **Variables, Expressions and Statements:**Values and data types, Variables, Variablenamesandkeywords,Statements,Evaluatingexpressions,
  Operators and operands, Type converter functions, Order of operations, Operations on strings, Input, Composition, The modulus operator.
- **Iteration:**Assignment, updating variables, the for loop, the while statement, the Collatz 3n + 1 sequence, tables, two-dimensional tables, break statement, continue statement, paired data, Nested Loops for Nested Data.
- **Functions:** Functions with arguments and return values.

Chapters:1.1-1.7,2.1-2.12,3.3,4.4,4.5

Text Book: Peter Wentworth, Jeffrey Elkner, Allen B. Downey and Chris Meyers- How to think like a computer scientist: learning with python 3. Green Tea Press, Wellesley, Massachusetts,2020

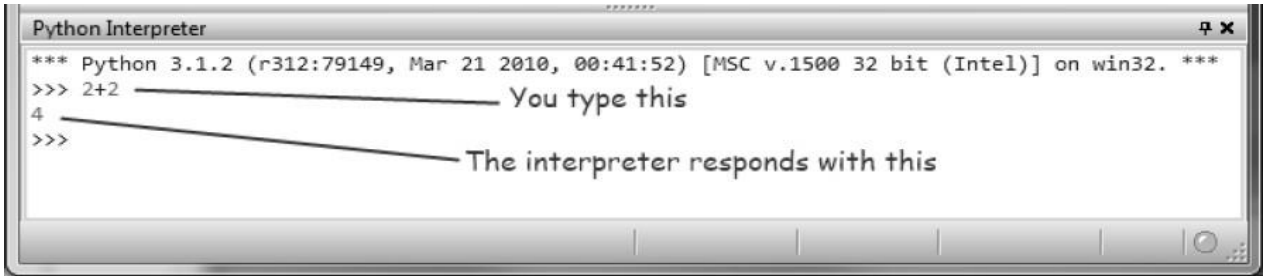
Chapter1:TheWayofthe Program

The Python programming language: Python is a general-purpose, interpreted, object-oriented, powerful high-level programming language.

- The engine that translates and runs Python is called the Python Interpreter. The Python interpreter can be used in TWO primarymodes

1. **Immediate mode:**The interpreter processes the input and displays the result or any output immediately after execution.
2. **Script Mode:**This mode refers to the method of executing Python codeby runningasavedfilecontainingthecode. Sucha fileiscalleda script. Scripts have the advantage that they can be saved to disk, printed, and so on.

Dr.sureshaV



The>>> is called the Python prompt. The interpreter uses the prompt to indicate that it is ready for instructions. If you typed2 + 2, and the interpreter evaluated our expression, and replied 4, and on the next line it gave a new prompt, indicating that it is ready for more input.

- **Pythoninterpretermodes**

	ImmediateMode	**ScriptMode**
Type	Itisalsocalledtheinteractive modetype	Non-interactivemodetype
Execution Style	Codes are executed line by line, and the results or errors are displayed immediately after each line is entered.	Python code is written in a file and then executed as a whole program.Theinterpreterreads andexecutestheentirescript fromtoptobottom.
UseCases	Ideal for learning Python, testing small code snippets, exploring the functionalities of libraries,performingquick calculations,anddebugging	Essential for developing larger applications, creating reusable modules, automating tasks, andbuildingstructured programs.
Characteristics	• Provides immediate feedback. • Commands are not saved persistently. • Limited for developing large, complex programs.	• Code is saved persistently in files, allowing for easy modification, version control, and sharing. • Facilitates organised and structured programming with functions, classes, and modules. • Output is displayed after the entire script or relevant sections have been executed. • Provides better support for debugging and testing oflarger programs.

Insummary:

- **Interactivemode:** isforimmediateexecutionandexperimentation, offering quick feedback.
- **Script mode:** is for developing and running complete, structured programs, providing persistence and scalability.

Dr. Sireesha V

- **Pythonlanguagefeatures/Advantages/Characteristics*****

1. **Simple and Readable Syntax:** Python's syntax is clear and easy to read, making it an ideal language for both beginners and experienced programmers.
2. **Rich Standard Library:**Python has several standard libraries that provide ready-to-use modules and functions for various tasks.
3. **High-level Language:** It is easier for humans to understand and write. This nature makes Python suitable for rapid development and prototyping.
4. **Dynamically Typed:** The data types of variables are determined during run-time. We do not need to specify the data type of a variable during writing code.
5. **Object-Oriented Programming (OOP) Support:** PythonsupportsOOP paradigms, allowing developers to create reusable and modular code through classes and objects, encapsulation, inheritance, etc.
6. **Interpreted Language:** It means that the code is executed line by line without the need for compilation. This enables rapid development cycles and facilitates interactive programming.
7. **Integrated Language:**We can easily integrate it with other languages such as C, C++, etc.
8. **Cross-platform Compatibility:**Python can be easily installed on Windows, macOS, and various Linux distributions, allowing developers to create software that runs seamlessly across different operating systems.
9. **Open source and free:**It is an open-source language, which means that anyone can download it, use it, and share it. Moreover, it is free of cost.
10. **Garbage Collected:**Memory allocation and deallocation are automatically managed. Programmers do not specifically need tomanage the memory.
11. **Supports GUI programming:**Python has support for creating various GUI applications. It is the most popular for making graphical apps.

Whatisaprogram?

- A program is a sequence of instructions that specifies how to perform a computation.
- The computation might be something mathematical, logical or othertypes of operations. Example: solving a system of equations or finding the roots of a polynomial, etc.
- Every programming language includes input, output, maths, conditional executions and repetition functions, etc.

1.3Whatisdebugging?

- The systematic process of finding, isolating and correcting the error inthe programming is called debugging. Programming errors are called bugs
- Threekindsoferrorscanoccurinaprogram:
    1. **Syntaxerrors**
    2. **Runtimeerrors**
    3. **Semanticerrors**

Syntaxerrors:

- Syntax refers to the structure of a program and the rules about that structure.
- A syntax error in programming refers to a mistake in the source codethat violates the grammatical rules or structure of the specific programming language being used.
- This prevents the compiler or interpreter from understanding and executing the code.
- If there is a single syntax error anywhere in your program, Itwill display an errormessage and quit, and youwill not beable torun your program.
- Commoncausesofsyntaxerrorsinclude:
    1. **Missing or misplaced punctuation:** This can include missing semicolons, parentheses, brackets, or quotation marks.
    2. **Incorrectly spelt keywords or variable names:** Programming languages have specific keywords (e.g., if,for,while) that must be spelt correctly.
    3. **Improper indentation:** In Python, indentation is part of the syntax, and incorrect indentation will lead to errors.

Dr. Suresha V

4. **Using incorrect operators or symbols:** Forexample,using=for comparison instead of == in many languages.
5. **Unmatched opening and closing elements:** Such as a missing closing parenthesis or bracket, etc.

Runtimeerrors:

- Aruntimeerrorinprogrammingisanerrorthatoccursduringthe execution of a program, after it has been successfully compiled.
- Runtimeerrorsmanifestonlywhentheprogramisactuallyrunning.
- These errorsare alsocalled exceptionsbecause they usually indicatethat something exceptional (and bad) has happened.
- Commoncausesofruntimeerrorsinclude:
  1. **Division by Zero:** Attempting to divide a number by zero is mathematically undefined.
  2. **Array Index Out of Bounds**:Accessing an element inan array using an index that is outside the valid range of indices for that array.
  3. **Infinite Loops or Recursion**: A loop or recursive function that never terminates, leading to resource exhaustion, etc.

Semanticerrors:

- A semantic error in programming occurs when code is syntactically correct and can be successfully compiled or interpreted. Still, it does not produce the intended or expected results due to a wrong in its logic or meaning.
- Identifying semantic errors can be tricky because it requires you to work backwards by looking at the output of the program. And trying to figure out what it is doing.
- Examplesofsemanticerrorsinclude:
  1. Incorrect mathematical operations: Using subtraction instead of addition, or multiplication instead of division, leading to incorrect calculations.
  2. Logical flaws: An algorithm designed to sort data might sort it in descending order when ascending order was intended.
  3. Incorrect variable assignments or usage: Assigning a value to the wrong variable or using a variable before it has been properly initialised, etc.

Dr. Suresha V

Experimentaldebugging

- Debugging is alsolike an experimental science. The most important skills you will acquire are debugging.Once you have an idea of what is going wrong, you modify your program and try again.
- Although debugging can be frustrating, it is one of the most intellectually rich, challenging, and interesting parts of programming.
- In some ways, debugging is like detective work. You are confronted with clues, and you have to infer the processes and events that led to the results you see.
- If your hypothesis was correct, then you can predict the result of the modification, and you take a step closer to a working program. If your hypothesis was wrong, you have to come up with a new one.
- KeyStepsinExperimentalDebugging
  a. **Observe and Gather Information:** Note the observed error or unexpected behaviour in the software.
  b. **Formulate a Hypothesis:** Based on the observations, find out the root cause of the error.
  c. **Make Predictions:** Determine what should happen if your hypothesis is correct after a specific modification.
  d. **TesttheHypothesis:** Make a small change to the code, implementing the predicted modification.
  e. **Analyse the Results:** Observe the outcome of the change. If it matches your prediction, you're closer to a solution.If not, refine your hypothesis or form a new one based on the new information.
  f. **Repeat:** Continue this cycle of modification and observation until the error is identified and fixed.

Dr.Suresha V

Chapter2:PythonVariables,ExpressionsandStatements

ValuesandDataTypes:

- **Values:**
  - Valuesarethefundamentalbuildingblocksofinformationwithina program's execution.
  - In programming languages, a value represents a piece of data that canbe stored, manipulated, and processed by a program.
  - Examples:4,`HiGoodMorning',etc
- **Datatypes:**
  - A data type isa category for values,and every value belongs toonedata type. In programming, a datatype is an important concept.
  - Sometypesofdataareasfollows,withanexample

1. Integertype(int):

- ItisanumericdatatypeinPythonusedtoholdnumericvalues.
- Theinteger(orint)datatypeindicatesvaluesthatarewholenumbers.
- Itisrepresentedasintanditholdssignedintegersofnon-limitedlength.
- Forexample,int:2,3,0,-13,78, -132,etc.

2. Floattype(float):

- ItisalsoanumericdatatypeinPythonusedtoholdnumeric values.
- Thefloat(orfloat)datatypeindicatesvaluesthataredecimalnumbers.
- Itisrepresentedasafloatanditholdsfloatingdecimalpointsandit's accurate upto 15decimal places. ExFloat:2.345, -0.234,5.0,1.23, etc..

3. StringType(str):

- Pythonprogramscanalsohavetextvaluescalledstrings,or str.
- StringsinPythonaresurroundedbyeithersinglequotationmarksor double quotation marks. Ex: 'Hello' is the same as "Hello".
- Thestringwithnocharacters,'',iscalledablank string.
- Thedatatypecanbeidentifiedofanyobjectbyusing**type()**function
- Example 1:

```
>>> type("Hello, World!")
<class 'str'>
>>> type(17)
<class 'int'>
```

Dr. Suresh V

Example2	Compliedoutput
x='Hi 1st sem students' y= 2024 z= 20.345 print(type(x)) print(type(y)) print(type(z))	<class 'str'> <class 'int'> <class 'float'>

- Example3:Thevalueslike"17"and"3.2". Theylook likenumbers,but they are in quotation marks like strings.

```
>>> type("17")
<class 'str'>
>>> type("3.2")
<class 'str'>
```

- Example 4: Strings in Python can be enclosed in either single quotes (') or double quotes ("), or three of each (''' or """)

```
>>> type('This is a string.')
<class 'str'>
>>> type("And so is this.")
<class 'str'>
>>> type("""and this.""")
<class 'str'>
>>> type('''and even this...''')
<class 'str'>
```

- Example 5: When using large integers with commas between groups of three digits, as in 42,000 is not a legal integer in Python, but it does mean something else, which is legal

```
>>> 42000
42000
>>> 42,000
(42, 0)
```

- In the above, that's not what we expected at all! Because of the comma, Python chose to treat this as a pair of values. Hence not to put commas or spaces in your integers, no matter how big they are.
- In formal languages are strict, the notation is strict, and even the smallest change might mean something quite different from what you intended.

Variables

- Variablesarepicesofcomputer'smemoryforstoringdatavaluesthatcanbe changed or manuplate.

- VariableinProgrammingisanamedstoragelocationthatholdsavalueordata.
- TheassignmentstatementgivesavaluetoavariableThis statement is used to assign the value to the specified variable. An assignment statement consists of a variable name, an equal sign (called the assignment operator), and the value to be stored.
- Example:

```
>>> message = "What's up, Doc?"
>>> n=17
>>> pi=3.14159
```

- In above example makes three assignments. The first assigns the string value "What's up, Doc?" to a variable named message. The second gives theinteger17ton,andthethirdassignsthefloating-pointnumber 3.14159 to a variable called pi.

- **Overwritingthevariable:**Itreferstotheprocessofassigninganew value to an existing variable, thereby replacing its previous value. example

```
❶ >>> spam = 40
  >>> spam
  40
  >>> eggs = 2
❷ >>> spam + eggs
  42
  >>> spam + eggs + spam
  82
❸ >>> spam = spam + 2
  >>> spam
  42
```

❶ A variable is initialized the first time a value is stored in it.

❷. After that, use it in expressions with other variables and values.

③ When a variable is assigned a new value, the old value is forgotten, which is why spam is evaluated to 42 instead of 40 at the end.

VariablenamesandKeywords

- **Variable names :** A variable can have a short name (like x and y) or a more descriptive name (age, carname, total_volume). We can name a variable anything as long as it obeys the following rules:
  1. Itcanbeonlyoneword.
  2. Itcan'tbeginwithanumber.
  3. Itcanuseonlyletters,numbers,andtheunderscore(_)character.
  4. Onlycontainalpha-numericcharactersandunderscores(A,0-9, and _)
  5. Variable names are case-sensitive (age, Age & AGE are 3 different variables)

- FollowingTable1-3:givesexamplesofvalidandinvalidvariablenames

Valid variable names	Invalid variable names
balance	current-balance (hyphens are not allowed)
currentBalance	current balance (spaces are not allowed)
current_balance	4account (can't begin with a number)
_spam	42 (can't begin with a number)
SPAM	total_$um (special characters like $ are not allowed)
account4	'hello' (special characters like ' are not allowed)

- **Python Kewords:**Keywords in Python are reserved words that have special meaning and purpose within the language. They are fundamental building blocks that define the syntax and structure of Python programs. "A variable name cannot be any of the Python keywords".
- Pythonhasthirty-somethingkeywords.

and	as	assert	break	class	continue
def	del	elif	else	except	exec
finally	for	from	global	if	import
in	is	lambda	nonlocal	not	or
pass	raise	return	try	while	with
yield	True	False	None		

Statements

- In Python, a statement is an **instruction** that the Python interpreter can execute to perform a specific action.
- Whenyoutypeastatementonthecommandline,Pythonexecutesit. Statements don't produce any result.
- CommontypesofstatementsinPythonare:
  1. AssignmentStatements2.ExpressionStatements
  3.ConditionalStatements 4.LoopingStatements
  5.ControlFlowStatements7.ImportStatementsetc.

Evaluatingexpressions:

- An expression is a combination of values, variables, operators, and calls to functions.
- IfyoutypeanexpressionatthePythonprompt,theinterpreterevaluates it and displays the result. Example

```
>>> 1 + 1
2
>>> len("hello")
5
```

- Intheaboveexample,lenisabuilt-inPythonfunctionthatreturnsthe number of characters in a string.
- The evaluation of an expression produces a value, which is why expressionscanappearonthe right-handsideofassignmentstatements. A value all by itself is a simple expression, and so is a variable. Example

```
>>> 19
19
>>> y=3.141
>>> x=len('kvgce')
>>> x
5
>>> y
3.141
```

2.6Operatorsandoperands

- Operatorsarespecialsymbolsorkeywordsthatperformoperationson valuesorvariables.
- Thesevaluesorvariablesareknownasoperands.
- Operatorsindicatethespecificactiontobeperformed,suchasaddition, comparison, or logical evaluation.
- Examplesofoperatorsinclude:
  - **Arithmetic operators**:+(addition),-(subtraction),* (multiplication), / (division), ** (exponentiation).
  - **ComparisonOperators**:==(equalto),!=(notequalto),>(greater than), < (less than).
  - **LogicalOperators**:and,or,not.
  - **AssignmentOperators:**=(assignment),+=(addandassign).
- Operandsarethevaluesorvariablesuponwhichanoperatoracts.They

Dr. Suresha V

arethedatathattheoperationmanipulates.

- For example, in the expression **x + 5:** + is the operator (addition) and x and 5 are the operands.
- Operands can be various data types, including numbers, strings, booleans, or even other expressions.The combination of operators and operands forms expressions, which are then evaluated to produce a result.
- Someoftheoperatorsshown below

Operator	Name	Example
+	Addition	x + y
-	Subtraction	x - y
*	Multiplication	x * y
/	Division	x / y
%	Modulus	x % y
**	Exponentiation	x ** y
//	Floor division	x // y

Examples:

```
>>> 2 ** 3
8
>>> 3 ** 2
9
```

```
>>> minutes = 645
>>> hours = minutes / 60
>>> hours
10.75
```

```
>>> 7 / 4
1.75
>>> 7 // 4
1
>>> minutes = 645
>>> hours = minutes // 60
>>> hours
10
```

Note:*Division,**Exponential,/Division,//floordivision(thefloor division // rounds the result down to the nearest whole number)

Typeconverterfunctions

- Pythonprovidesseveralbuilt-infunctionsforexplicittypeconversion, also known as typecasting. Some examples `int, float and str,`
- Thesefunctionsallowaprogrammertointentionallychangethedata type of a value. Common Type Conversion Functions:

1. **int():** Convertsavaluetoan integer.
   - Itcanconvertfloats(truncatingthedecimalpart),numericstrings, andbooleans(Truebecomes1,Falsebecomes0).
   - Example:1

```
>>> int(3.14)
3
>>> int(3.9999)            # This doesn't round to the closest int!
3
>>> int(3.0)
3
>>> int(-3.999)            # Note that the result is closer to zero
-3
>>> int(minutes / 60)
10
>>> int("2345")            # Parse a string to produce an int
2345
>>> int(17)                # It even works if arg is already an int
17
```

   - Example 2:

```
>>> int('123 bottles')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: invalid literal for int() with base 10: '123 bottles'
```

2. **float():**Convertsavaluetoafloating-pointnumber.
   - Itcanconvertintegers,numericstrings,andbooleans(True becomes1.0,Falsebecomes0.0).
   - Example:float(5)resultsin5.0;float("2.718")resultsin2.718.
3. **str():**Convertsanydatatypetoitsstringrepresentation.
   - Example:str(42)resultsin"42";str([1,2,3])resultsin"[1,2,3]".

Orderofoperations

- When more than one operator appears in an expression, the order of evaluation depends on the rules of precedence.
- Python follows the standard order of operations, often remembered by the acronym PEMDAS or BEDMAS. This dictates the sequence in which operations are evaluated within an expression

- Python follows the same precedence rules for its mathematical operators that mathematics does.
- TheorderofoperationsinPython,fromhighesttolowestprecedence,is:

1. **Parentheses ():** Operationswithinparenthesesarealwaysevaluated first, regardless of the operators inside
   - `Example:result=(5+3)*2 #(5+3)isevaluatedfirst, thenmultipliedby2.Similarly2*(3-1)is4,and(1+1)**(5-2)is8.`
2. **Exponents**:**Exponentiation is evaluated after parentheses. Exponentiation has the next highest precedence**,**
   - `Example:2**1+1is3andnot4,and3*1**3is3andnot27.`
3. **Multiplication and both Division operators:**Both have the same precedence, which is higher than Addition and Subtraction, which also havethesameprecedence.So2*3-1yields5ratherthan4,and5-2*2is 1, not 6.
1. **Operators with the same precedence are evaluated from left-to-right**:In algebra we say theyare left-associative.So in the expression6-3+2, the subtraction happens first, yielding 3. We then add 2 to get the result 5. If the operations had been evaluated from right to left, the result would have been 6-(3+2), which is 1.

   Examples:

```
>>> 2 + 3 * 6
20
>>> (2 + 3) * 6
30
>>> 48565878 * 578453
28093077826734
>>> 2 ** 8
256
>>> 23 / 7
3.2857142857142856
>>> 23 // 7
3
>>> 23 % 7
2
>>> 2     +           2
4
>>> (5 - 1) * ((7 + 1) / (3 - 1))
16.0
```

```
(5 - 1) * ((7 + 1) / (3 - 1))
        ↓
    4 * ((7 + 1) / (3 - 1))
        ↓
    4 * (  8  ) / (3 - 1))
        ↓
    4 * (  8  ) / (  2  )
        ↓
    4 * 4.0
        ↓
    16.0
```

Operationsonstrings

- In general, you cannot perform mathematical operations on strings, even if the strings look like numbers.The following are illegal

```
>>> message - 1          # Error
>>> "Hello" / 123        # Error
>>> message * "Hello"    # Error
>>> "15" + 2             # Error
```

- Interestingly, the + operator does work with strings, but for strings, the + operator represents concatenation, not addition.
- Stringconcatenation and replication operation can be performed. Concatenationistheprocessofjoiningtwoormorestringstogetherand formingonesinglestring.
- **Example**:

```
>>> 'hello' +' Good morning'
'hello Good morning'
```

- The * operator also works on strings; it performs repetition. For example, 'Fun'*3 is'FunFunFun'. One of the operands has to be a string; the other has to be an integer.
- Replicating the same string "n" several times is called string replication. Use the * operator on a string to repeat the string a provided number of times.
- Thesyntaxfortheoperationis:**result='string'*integernumber**
- **Example:'Kvg'*3,results'kvgKvgKvg'**

Input

- Theinput() function allows takes a user input. By default, it returns theuser input in the form of a string.
- The input() function waits for the user to type some text on the keyboard and press ENTER.

Example1:

```
>>> name=input('Enter your Name=')
 Enter your Name=
```

Example2:

```
>>> Name=input()
```

Dr. Suresha V

Composition

- Oneofthemostusefulfeaturesofprogramminglanguagesistheirability to take small building blocks and compose them into larger chunks.
- We know that, the elements of a program — variables, expressions, statements, and function calls — in isolation, without talking about how to combine them.
- For example, consider a small four-step program that asks the user to inputavalue forthe radiusofa circle,andthencomputes theareaofthe circle from the formula$\boldsymbol{A = \pi r^2}$
- Firstly,we'lldothefourstepsoneata time:

```
response = input("What is your radius? ")
r = float(response)
area = 3.14159 * r**2
print("The area is ", area)
```

- Nowlet'scomposethefirsttwolinesintoasinglelineofcode,and composethesecondtwolinesintoanotherlineofcode.

```
r = float( input("What is your radius? ") )
print("The area is ", 3.14159 * r**2)
```

- Ifwereallywantedtobetricky,wecouldwriteitallinone statement:

```
print("The area is ", 3.14159*float(input("What is your radius?"))**2)
```

2.12Themodulusoperator

- The modulus operator works on integers (and integer expressions) and gives the remainder when the first number is divided by the second.
- In Python, the modulus operator is a percent sign (%). The syntax is the same as for other operators. It has the same precedence as the multiplication operator.

```
>>> q = 7 // 3       # This is integer division operator
>>> print(q)
2
>>> r  = 7 % 3
>>> print(r)
1
```

- Themodulusoperatorusefultocheckwhetheronenumberisdivisibleby another, conversion and extract right most digit.

Chapter3:ProgramFlow-Iteration

Iteration:

- Iteration in Python refers to the process of repeatedly executing a block of code for each item in a sequence or until a certain condition is met.
- It is a fundamental concept in programming for processing collections of data or performing repetitive tasks.

Assignment

- Python'sassignmentoperatorsareusedtoassignvaluestovariables.
- Themostfundamentalisthesimpleassignmentoperator, =,which assigns the value on its right to the variable on its left.
- Anewassignmentmakesanexistingvariablerefertoanewvalue.
- Example:

Program	Output
1 x=17 2 print(x) 3 x=5 4 print(x)	17 5

- It is especially important to distinguish between an assignmentstatement and equality statement.
- Because Python uses the equal sign (=) for assignment and equality operator is ==.
- Note that an equality test is symmetric, but assignment is not. For example, if a == 7 then 7 == a. But in Python, the statement a = 7 is legal and 7 = a is not.
- In Python, an assignment statement can make two variables equal, but because further assignments can change either of them, they don't have to stay that way, example

```
1 a = 5
2 b = a    # After executing this line, a and b are now equal
3 a = 3    # After executing this line, a and b are no longer equal
```

- The third line changes the value of a but does not change the value of b, so they are no longer equal.

Dr.Sur-sha V

Updatingvariables

- Updating variables in Python involves changing the value that a variable refers to. This is primarily done through assignment(=) statements.
- When an assignment statement is executed, the right-hand side expression is evaluated first. This produces a value. Then the assignment is made, so that the variable on the left-hand side now refers to the new value. Example

```
1 n = 5
2 n = 3 * n + 1
```

- Inabove program, Line2 means getthe currentvalue of n, multiply it by three and add one, and assign the answer to n, thus making n refer to thevalue. Soafterexecutingthetwolinesabove,nwillpoint/refertothe integer 16.
- If you try to get the value of a variable that has never been assigned to, you'll get an error:

```
>>> w = x + 1
Traceback (most recent call last):
  File "<interactive input>", line 1, in
NameError: name 'x' is not defined
```

- Before you can update a variable, you have to initialize it to some rting value, usually with a simple assignment

```
1 runs_scored = 0
2 ...
3 runs_scored = runs_scored + 1
```

- Line 3—updating a variable by adding 1 to it—is very common. It iscalled an increment of the variable; subtracting 1 is called a decrement

---

Theforlooprevisited

- Aforloopisusedforiteratingoverasequence.
- Withtheforloopwecanexecuteasetofstatements,onceforeachitem in a list, tuple, set etc.
- Incode,aforstatementalwaysconsistsofthefollowing:

1.The forkeyword 2.Avariablename3.TheinKeyword
4.Acallontherange5.AColon

- The for loop processes each item in a list. Each item in turn is (re-)assigned to the loop variable, and the body of the loop is executed
- **Example1:**

```
fruits=["apple","banana","cherry"] for x
  in fruits:
    print(x)
output=
```

```
apple
banana
cherry
```

- **Example2:**

```
forfriendin["Joe","Zoe","Zuki","Thandi","Paris"]:
    invite="Hi"+friend+".Pleasecometomyparty!"print(invite
    )
```

Output:

```
Hi Joe. Please come to my party!
Hi Zoe. Please come to my party!
Hi Zuki. Please come to my party!
Hi Thandi. Please come to my party!
Hi Paris. Please come to my party!
```

Runningthroughalltheitemsinalistiscalledtraversingthelist,ortraversal.

3.3.4Thewhilestatement

- PythonWhileLoopisusedtoexecuteablockofstatementsrepeatedly untilagivencondition issatisfied. When the conditionbecomes false, the lineimmediatelyaftertheloopintheprogramisexecuted.
  - Incode,awhilestatementalwaysconsistsofthefollowing:

1. Thewhilekeyword
2. Acondition(thatisanexpressionthatevaluatestoTrueorFalse.
3. Acolon
4. Startingonthenextline,anindentedblockofcode(calledthe while clause)

- Syntax:forawhileloopstatement

```
while <CONDITION>:
    <STATEMENT>
```

- **Example1:considertheprogram**

```
1  n = 6
2
3  current_sum  = 0
4  i = 0
5  while i <= n:
6      current_sum += i
7      i += 1
8  print(current_sum)
```

- Descriptionofflowofexecutionforawhilestatement:
  - Evaluatetheconditionatline5,yieldingavaluewhichiseitherForT.
  - IfthevalueisFalse, exit thewhile statementandcontinue execution at the next statement (line 8 in this case).
  - IfthevalueisTrue,executeeachofthestatementsinthebody(lines6 and 7) and then go back to the while statement at line 5.
- Thebodyconsistsofallofthestatementsindentedbelowthewhile keyword.

3.3.5TheCollatz3n+1sequence

- TheCollatz3n+1sequence,alsoknownastheCollatzconjecture,isa mathematical sequence defined for positive integers.
- Therulesforgeneratingthesequenceareasfollows:
  - Ifthecurrentnumberniseven,thenextnumberinthesequence is n / 2.
  - Ifthecurrentnumber nisodd,thenextnumberinthesequence is 3n + 1.
  - The conjecture states that this sequence will eventually reach thenumber 1 for any positive integer starting value.
- ThisPythoncodecapturesthisalgorithm:

```
1  n = 1027371
2
3  while n != 1:
4      print(n, end=", ")
5      if n % 2 == 0:          # n is even
6          n = n // 2
7      else:                   # n is odd
8          n = n * 3 + 1
9  print(n, end=".\n")
```

Dr.Sureshna V

- Notice first that the print function on line 4 has an extra argumentend=",". This tells the print function to follow the printed string with whatever the programmer chooses (in this case, a comma followed by a space), instead of ending the line.
- So each time something is printed in the loop, it is printed on the same output line, with the numbers separated by commas.
- Thecalltoprint(n,end=".\n")at line9 aftertheloopterminateswillthen print the final value of n followed by a period and a newline character.
- Example 2:

```
n = 20

while n != 1:
    print(n, end=", ")
    if n % 2 == 0: # n is even
        n = n // 2
    else: # n is odd
        n = n * 3 + 1
print(n, end=".\n")
```

Output:

```
20, 10, 5, 16, 8, 4, 2, 1.
>>>
```

3.3.6Tracingaprogram

- Towriteeffectivecomputerprograms,andtobuildagoodconceptualmodel ofprogramexecution,aprogrammerneedstodeveloptheabilitytotrace theexecutionofacomputer program.
- Tracing involves becoming the computer and following the flow of execution through a sample program run, recording the state of all variables and any output the program generates after each instruction is executed.
- ConsidertheCollatz3n+1sequencewithn=3

```
n = 3
while n != 1:
    print(n, end=", ")
    if n % 2 == 0: # n is even
        n = n // 2
    else: # n is odd
        n = n * 3 + 1
print(n, end=".\n")
```

Outputsequence:

```
n               output printed so far
--              --------------------
3               3,
10              3, 10,
5               3, 10, 5,
16              3, 10, 5, 16,
8               3, 10, 5, 16, 8,
4               3, 10, 5, 16, 8, 4,
2               3, 10, 5, 16, 8, 4, 2,
1               3, 10, 5, 16, 8, 4, 2, 1.
```

3.3.9Tables

- One of the things loops are good for is generating tables. For some operations, computersusetablesofvaluestoget anapproximateanswer and then perform computations to improve the approximation.
- The following program outputs a sequence of values in the left column and 2 raised to the power of that value in the right column. Example

```
1 for x in range(13): # Generate numbers 0 to 12
2     print(x, "\t", 2**x)
```

- Output:

```
0       1
1       2
2       4
3       8
4       16
5       32
6       64
7       128
8       256
9       512
10      1024
11      2048
12      4096
```

  - Because of the tab characters between the columns, the position of the second column does not depend on the number of digits in the first column.

3.3.10Two-dimensionaltables

- A two-dimensional table is a table where you read the value at the intersection of a row and a column A multiplication table is a good example.
- Let'ssaytoprintamultiplicationtableforthevaluesfrom1to6.
- A good way to start is to write a loop that prints the multiples of 2, all on one line, example

```
1 for i in range(1, 7):
2     print(2 * i, end="    ")
3 print()
```

- abovewe'veusedtherange function,but made it start itssequenceat1. As the loop executes, the value of i changes from 1 to 6.
- When all the elements of the range have been assigned to i, the loop terminates. Each time through the loop, it displays the value of 2 * i, followed by three spaces.
- Again, the extra end="" argument in the print function suppresses the newline, and uses three spaces instead.
- After the loop completes, the call to print at line 3 finishes the current line, and starts a new line.
- Theoutput oftheprogramis:

```
2       4       6       8       10      12
```

3.3.11Thebreakstatement

- 'Break' in Python is a loop control statement. It is used to control the sequence of the loop.
- The break keyword is used to break out a for loop, or a while loop. If the execution reaches a break statement, it immediately exits the while loop's clause.
- Incode,abreakstatementsimplycontainsthebreakkeyword.

Example

End the loop if i is larger than 3:

```
for i in range(9):
  if i > 3:
    break
  print(i)
```

```
0
1
2
3
```

```
i = 1
while i < 9:
  print(i)
  if i == 3:
    break
  i += 1
```

```
1
2
3
```

Dr. Nireesha V

```
for i in [12, 16, 17, 24, 29]:
    if i % 2 == 1:  # If the number is odd
        break       #  ... immediately exit the loop
    print(i)
print("done")
```

This prints:

```
12
16
done
```

Thecontinuestatement

- This is a control flow statement (continue keyword) isused to end the current iteration in a for loop (or awhile loop) and continues to the next iteration. It is used inside the loop.
- When the program execution reaches a continue statement, the program execution immediately jumps back to the start of the loop and reevaluates the loop's condition. Example 1

```
for i in [12, 16, 17, 24, 29, 30]:
    if i % 2 == 1:       # If the number is odd
        continue         # Don't process it
    print(i)
print("done")
```

This prints:

```
12
16
24
30
done
```

Example 2:

Skip the iteration if the variable i is 3, but continue with the next iteration:

```
for i in range(9):
  if i == 3:
    continue
  print(i)
```

```
0
1
2
4
5
6
7
8
```

PairedData

- MakingapairofthingsinPythonisassimpleasputtingtheminto parentheses, like this:

```
year_born = ("Paris Hilton", 1981)
```

We can put many pairs into a list of pairs:

```
celebs = [("Brad Pitt", 1963), ("Jack Nicholson", 1937),
                                ("Justin Bieber", 1994)]
```

Here is a quick sample of things we can do with structured data like this. First, print all the celebs:

```
print(celebs)
print(len(celebs))
```

```
[("Brad Pitt", 1963), ("Jack Nicholson", 1937), ("Justin Bieber", 1994)]
3
```

- Noticethatthecelebslisthasjust3elements,eachofthempairs.Now weprintthenamesofthosecelebritiesbornbefore1980:

```
for name, year in celebs:
    if year < 1980:
        print(name)
```

```
Brad Pitt
Jack Nicholson
```

  - Thisdemonstratessomethingwehavenotseenyetintheforloop: insteadofusingasingleloopcontrolvariable,we'veusedapairof variablenames,(name,year),instead.
- The loop is executed three times — once for each pair in the list, and on each iteration both the variables are assigned values from the pair of data that is being handled.

NestedLoopsforNestedData

- Anestedloopmeansa loopstatement insideanotherloopstatement.For a nested loop, the inner loop performs all of its iterations for each iteration of the outer loop.
- Example:Iftheouterloopisrunningfromi=1 to5 andtheinnerloopis running from a to e. Then, for each value of the outer loop variable (i), the inner loop will run from a to e.

- NestedloopsinPython areused toiterateovernested datastructures, such as lists of lists, lists of dictionaries, or nested dictionaries.
- Theconceptinvolvesplacingoneloopinsideanother,wheretheinner loop executes completely for each iteration of the outer loop.
- Consideracase,wehavealistofstudents.Eachstudenthasaname which is paired up with another list of subjects that they are enrolled for:

```
students = [
    ("John", ["CompSci", "Physics"]),
    ("Vusi", ["Maths", "CompSci", "Stats"]),
    ("Jess", ["CompSci", "Accounting", "Economics", "Management"]),
    ("Sarah", ["InfSys", "Accounting", "Economics", "CommLaw"]),
    ("Zuki", ["Sociology", "Economics", "Law", "Stats", "Music"])]
```

- Here we've assigned a list of five elements to the variable students. Let's printouteachstudentname,andthenumberofsubjectstheyare enrolledfor:

```
# Print all students with a count of their courses.
for name, subjects in students:
    print(name, "takes", len(subjects), "courses")
```

- Pythonagreeablyrespondswiththefollowingoutput:

```
John takes 2 courses
Vusi takes 3 courses
Jess takes 4 courses
Sarah takes 4 courses
Zuki takes 5 courses
```

- Now we'd like to ask how many students are taking CompSci. This needs a counter, and for each student we need a second loop that tests each of the subjects in turn:

```
# Count how many students are taking CompSci
counter = 0
for name, subjects in students:
    for s in subjects:                    # A nested loop!
        if s == "CompSci":
            counter += 1

print("The number of students taking CompSci is", counter)
```

```
The number of students taking CompSci is 3
```

4.4Functionsthatrequirearguments

- In Python, functions can be defined torequirearguments, meaning that values must be provided for these arguments when the function is called.
- Most functions require arguments: For example, if we want to find the absolute value of a number, we have to indicate what the number is. Python has a built-in function for computing the absolute value.
- Example1:functionswithoneargument

```
>>> abs(5)
5
>>> abs(-5)
5
```

–Inaboveexample,theargumentstotheabsfunctionare5and-5.

- Example 2: Some functions take more than one argument.For example the built-in function pow takes two arguments, the base and the exponent. Inside the function, the values that are passed get assigned to variables called parameters.

```
>>> pow(2, 3)
8
>>> pow(7, 4)
2401
```

- Example3:Anotherbuilt-infunctionthattakesmorethanoneargument ismax.

```
>>> max(7, 11)
11
>>> max(4, 1, 17, 2, 12)
17
>>> max(3 * 11, 5**3, 512 - 9, 1024**0)
503
```

- max can be passed any number of arguments, separated by commas, and will return the largest value passed. The arguments can be either simplevaluesorexpressions.Inthelastexample,503isreturned,since it is larger than 33, 125, and 1.

4.5Functionsthatreturnvalues

- ThereturnstatementinPythonisusedwithinafunctiontoexitthefunction and send a value back to the caller.
- Example:

```
defcalculate_sum(a,b):
    """Thisfunctioncalculatesthesumoftwonumbers.""" result = a
    + b
    returnresult#Returnsthecalculatedsum
```

- In Python, functions can return values to the part of the code that called themusingthereturnstatement. Thisallowsthefunctiontoprovidea result or data back to the main program flow for further processing, assignment to variables, or use in other operations.
- A functionthatreturns avalueis called a fruitfulfunction. The opposite ofa fruitful function is a void function. It is not executed for its resulting value.
- Take an example to write as a fruitful function for compound interest, the standard formula

$$A = P\left(1 + \frac{r}{n}\right)^{nt}$$

Where,

- P = principal amount (initial investment)
- r = annual nominal interest rate (as a decimal)
- n = number of times the interest is compounded per year
- t = number of years

```
1  def final_amount(p, r, n, t):
2      """
3        Apply the compound interest formula to p
4         to produce the final amount.
5      """
6
7      a = p * (1 + r/n) ** (n*t)
8      return a          # This is new, and makes the function fruitful.
9
10 # now that we have the function above, let us call it.
11 toInvest = float(input("How much do you want to invest?"))
```

(continue

```
12 fnl = final_amount(toInvest, 0.08, 12, 5)
13 print("At the end of the period you'll have", fnl)
```

- The return statement is followed an expression (a in this case). This expression will be evaluated and returned to the caller as the "fruit" of calling this function.
- We prompted the user for the principal amount. The type of toInvest is a string, but we need a number before we can work with it. Because it is money, and could have decimal places, we've used the float type converter function to parse the string and return a float.
- Notice how we entered the arguments for 8% interest, compounded 12times per year, for 5 years.

Acknowledgement:

My sincere thanks to the author **Peter Wentworth, Jeffrey Elkner, Allen B. DowneyandChrisMeyers-,**becausetheabovecontentsarepreparedfrom there textbook**"How to think like a computer scientist: learning with python 3".** Green Tea Press, Wellesley, Massachusetts,2020



Preparedby:

Dr. Suresha V

Professor&Principal

Dept.ofElectronicsandCommunicationEngineering. Reach me at: suresha.vee@gmail.com

WhatsApp:+918310992434

Dr. Suresha V